Memoria en ADK — Estado del arte

Tema	Arquitectura, mecanismos y criterios de diseño de la memoria en el Agent Development Kit (ADK)
Versión de referencia	ADK **2.10.0** (Python)
Autor	Matías Zabaljáuregui
Convenciones	Lo afirmado sobre ADK proviene del código de 2.10.0 y su documentación; lo inferido sin ejecutar se marca *(inferido)*
Fecha	2026-10-05
Audiencia	Arquitectos e ingenieros que diseñan o revisan agentes con memoria en producción
Alcance	Contexto, sesión, estado y memoria de largo plazo; servicios incluidos en ADK; uso de embeddings en la recuperación y la consolidación; servicios propios; consideraciones de producción
Fuentes	Código fuente de ADK 2.10.0 revisado directamente; documentación oficial en adk.dev y de Memory Bank en Google Cloud

Resumen ejecutivo

En ADK, «memoria» no es un componente único sino una familia de mecanismos con alcances, costos y garantías muy distintos. Conviene separar cuatro capas: el **contexto** que recibe el modelo en una llamada, el **historial** de la sesión (`events`), el **estado** de la sesión (`state`) y la **memoria de largo plazo**, que vive en un servicio independiente (`BaseMemoryService`) y sobrevive entre conversaciones. Las dos primeras pertenecen a la sesión y responden a la pregunta «qué pasó en esta conversación»; la última responde a «qué sabemos de este usuario a partir de conversaciones anteriores».

El contrato de la memoria de largo plazo es deliberadamente mínimo: se ingiere una sesión (o un conjunto de eventos, o entradas ya elaboradas) y se busca por texto. Todo lo demás —qué se guarda, cómo se indexa, qué significa «relevante»— lo decide la implementación. ADK incluye tres: una en RAM para prototipos con búsqueda por palabras clave, el servicio administrado Memory Bank de Vertex AI, que extrae y consolida hechos con un modelo generativo y los recupera por similitud semántica, y un servicio basado en Vertex AI RAG que indexa transcripciones completas en un corpus vectorial. El contrato permite además escribir implementaciones propias.

Dos hechos de diseño condicionan casi todo lo demás. Primero, **ADK no persiste la sesión en la memoria por sí solo**: la ingesta debe pedirse explícitamente desde código o desde un callback. Segundo, **recuperar no es lo mismo que mostrar**: `search_memory` devuelve datos al código, y hace falta una herramienta (`PreloadMemoryTool`, `LoadMemoryTool` o una propia) para que el modelo vea el resultado.

Los **embeddings** son el mecanismo que vuelve útil a la memoria de largo plazo: convierten texto en vectores para comparar por significado y no por palabras. Intervienen en dos momentos distintos —al **leer**, para encontrar lo relevante, y, en los diseños con consolidación, al **escribir**, como filtro barato que decide cuándo vale la pena invocar a un modelo generativo—. La sección 5 les dedica un tratamiento detallado.

La decisión de diseño más importante es dónde ubicar el riesgo. Una memoria de turnos crudos desplaza el riesgo a la **lectura** (un dato guardado puede no llegar al modelo); una memoria de hechos consolidados lo desplaza a la **escritura** (un hecho mal extraído se guarda como si fuera verdad). Ninguna variante lo elimina; el diseño consiste en elegir cuál se está dispuesto a gestionar.

Índice

Resumen ejecutivo ... 1

1. Marco conceptual: cuatro capas de memoria ... 3

2. Arquitectura de ejecución ... 5
- 2.1 El Runner como punto de unión ... 5
- 2.2 El Context de herramientas y callbacks ... 5

3. Corto plazo: sesión y estado ... 6
- 3.1 La sesión y sus servicios ... 6
- 3.2 El estado y sus alcances ... 6
- 3.3 Control del historial y compactación ... 7
- 3.4 El ciclo de una llamada al modelo ... 7

4. Largo plazo: el servicio de memoria ... 10
- 4.1 El contrato ... 10
- 4.2 La escritura: qué dispara el guardado ... 10
- 4.3 La lectura ... 11

5. Embeddings en la memoria de largo plazo ... 12
- 5.1 Por qué una memoria necesita embeddings ... 12
- 5.2 Dónde aparecen los embeddings en ADK ... 12
- 5.3 El ciclo de los embeddings: escribir y leer ... 13
- 5.4 Consolidación: el triaje con embeddings ... 13
- 5.5 Consideraciones al incorporar embeddings propios ... 15

6. Los servicios de memoria incluidos en ADK ... 16
- 6.1 InMemoryMemoryService ... 16
- 6.2 VertexAiMemoryBankService ... 16
- 6.3 VertexAiRagMemoryService ... 17
- 6.4 Comparación ... 17

7. Cómo llega la memoria al modelo ... 18
- 7.1 PreloadMemoryTool: recuperación automática ... 19
- 7.2 LoadMemoryTool: el modelo decide ... 20
- 7.3 Lectura propia ... 20
- 7.4 Cómo elegir ... 20

8. Servicios de memoria propios ... 22
- 8.1 Almacén de turnos con índice vectorial ... 22
- 8.2 Almacén de hechos consolidados ... 22
- 8.3 Criterios de diseño que el contrato no impone ... 22

9. Patrones de uso y selección ... 24

10. Consideraciones de producción ... 27
- 10.1 Aislamiento y alcance ... 27
- 10.2 Privacidad, retención y borrado ... 27
- 10.3 Consistencia y latencia de escritura ... 27
- 10.4 Costo y tamaño del contexto ... 27
- 10.5 Observabilidad y pruebas ... 27
- 10.6 Evaluación de la calidad de recuperación ... 28

11. Glosario ... 29

12. Referencias ... 30

Apéndice. Método de verificación y límites ... 31

1. Marco conceptual: cuatro capas de memoria

La confusión más frecuente al diseñar agentes con memoria proviene de tratar bajo una sola palabra cosas que se comportan de manera muy diferente. ADK las separa con claridad, y el diseño empieza por respetar esa separación.

El **contexto de la petición** es lo que el modelo recibe en una única llamada: la instrucción del sistema, el historial relevante, el mensaje actual y cualquier contenido que las herramientas o los callbacks hayan agregado. Es efímero por definición: se arma antes de cada llamada y se descarta después. Todo lo demás es, en último término, una manera de decidir qué entra en ese contexto.

El **historial** es la lista cronológica de eventos de una sesión: mensajes del usuario y del agente, llamadas a herramientas y sus respuestas. Lo escribe el `Runner` mediante `append_event` y es lo que el modelo lee, por defecto, como conversación previa. Vive tanto como la sesión.

El **estado** es un diccionario de pares clave-valor asociado a la sesión, con una particularidad que suele pasarse por alto: **el modelo no lo lee**. Solo llega al modelo si una plantilla `{variable}` en la instrucción lo incorpora o si una herramienta lo devuelve como resultado. El estado es, por tanto, un almacén para el código de la aplicación, no una extensión del historial.

La **memoria de largo plazo** es un servicio externo a la sesión que indexa información por `(app_name, user_id)` y permite recuperarla mediante búsqueda. Es lo único de las cuatro capas que trasciende la conversación sin que el código de la aplicación lo gestione a mano. Su ingesta es explícita y su lectura, mediada por herramientas.

Dos criterios ordenan el resto del informe. El primero es la distinción entre **corto y largo plazo**: la sesión existe mientras dura esa conversación, la memoria existe entre conversaciones. El segundo, entre **escribir y leer**: qué entra en la memoria lo decide el código de la aplicación en el momento de la ingesta, y qué ve el modelo lo decide la herramienta de lectura en cada llamada. Separar ambas decisiones es lo que permite razonar sobre dónde puede fallar un diseño.

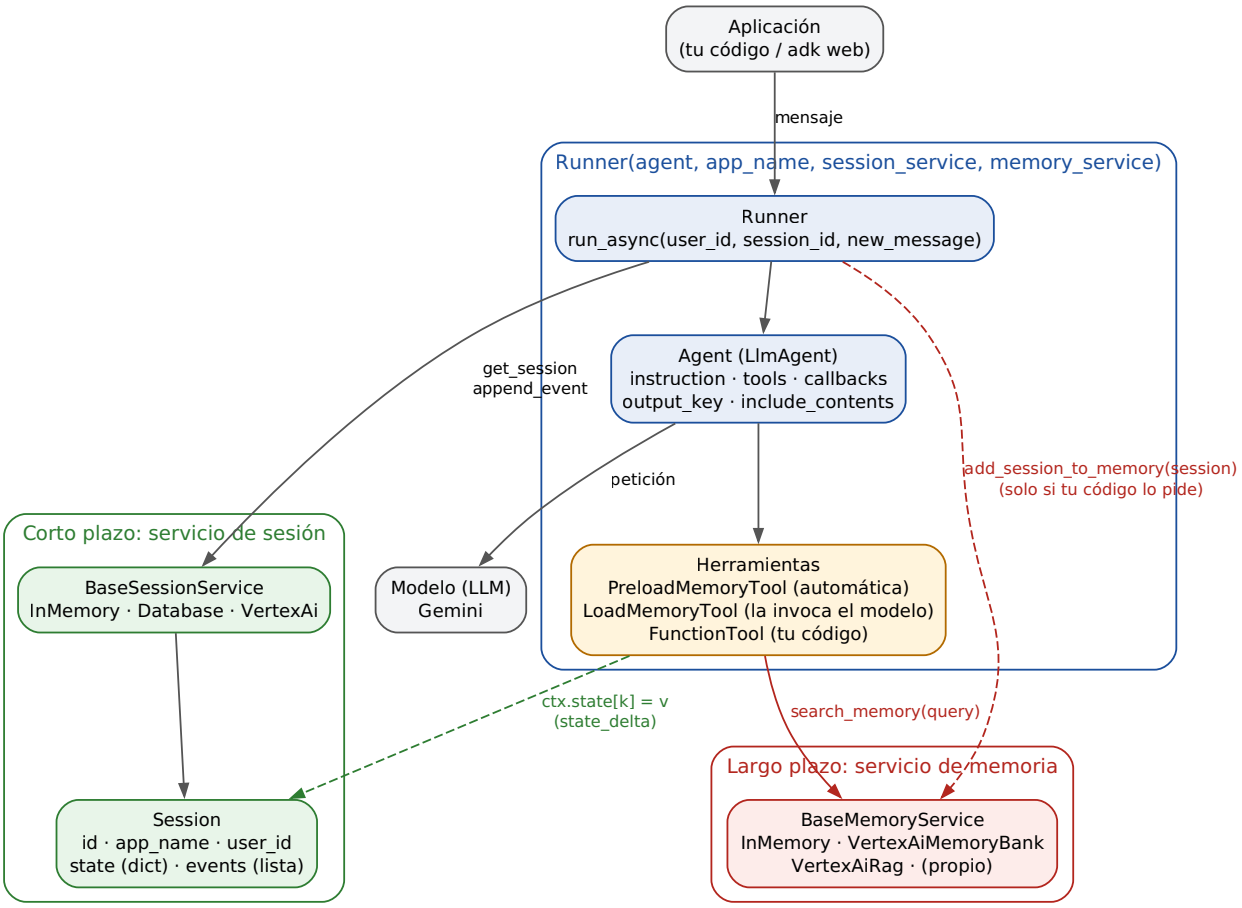


Figura 1 · Componentes: el Runner une agente, servicio de sesión y servicio de memoria; las herramientas leen la memoria y pueden escribir el estado.

Los artifacts (`BaseArtifactService`), que almacenan archivos versionados, son una pieza vecina que queda fuera del alcance de este informe.

2. Arquitectura de ejecución

2.1 El Runner como punto de unión

El `Runner` es el componente que reúne al agente con sus servicios. Recibe un servicio de sesión y, opcionalmente, un servicio de memoria, y los pone a disposición de todo lo que se ejecuta durante una invocación.

```python
from google.adk import Agent, Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import preload_memory

agente = Agent(name="asistente", model="gemini-2.5-flash",
               instruction="...", tools=[preload_memory])
runner = Runner(agent=agente, app_name="mi_app",
                session_service=InMemorySessionService(),
                memory_service=InMemoryMemoryService())
```

El agente no conoce el servicio concreto de memoria. Lo usa a través de herramientas y callbacks, que reciben un objeto `Context`. Esa indirección es lo que permite cambiar la implementación (de RAM a Memory Bank, por ejemplo) sin tocar la lógica del agente. Cuando se usa la línea de comandos de ADK (`adk web`, `adk api_server`), el servicio de memoria se selecciona con la opción `--memory_service_uri`, cuyos esquemas registrados son `memory://` (en RAM), `rag://` (Vertex AI RAG) y `agentengine://` (Memory Bank sobre un Agent Engine). Solo puede configurarse un servicio por `Runner`; si se necesitan varios almacenes, el segundo se instancia dentro del código de una herramienta, como indica la documentación oficial.

El par `(app_name, user_id)` define el alcance de la memoria. Que un usuario nunca vea la memoria de otro es una propiedad que garantiza **la implementación del servicio**, no el contrato; la sección 10.1 vuelve sobre esto.

2.2 El Context de herramientas y callbacks

Herramientas y callbacks reciben un `Context` que expone, para lo que aquí interesa, el estado (`state`, con seguimiento de cambios), la sesión actual (`session`) y cuatro operaciones de memoria: `search_memory(query)`, `add_session_to_memory()`, `add_events_to_memory(events, custom_metadata)` y `add_memory(memories, custom_metadata)`. La búsqueda toma `app_name` y `user_id` de la invocación en curso, de modo que el código de la herramienta no necesita (ni puede equivocarse en) el alcance. Las tres operaciones de escritura lanzan `ValueError` si el `Runner` no tiene servicio de memoria configurado.

3. Corto plazo: sesión y estado

3.1 La sesión y sus servicios

Una `Session` contiene su `id`, `app_name`, `user_id`, el `state`, la lista de `events` y la marca `last_update_time`. El contrato `BaseSessionService` define la creación, lectura, listado y borrado de sesiones, además de `append_event`, que es el único camino legítimo para modificar el historial y el estado de forma persistente.

ADK ofrece tres servicios. `InMemorySessionService` guarda todo en RAM y es adecuado para desarrollo y pruebas, porque se pierde al terminar el proceso. `DatabaseSessionService` persiste en una base de datos relacional a partir de una URL de conexión. `VertexAiSessionService` delega el almacenamiento en un Agent Engine de Vertex AI. La lectura admite una configuración (`GetSessionConfig`) que limita los eventos devueltos por cantidad reciente o por marca temporal, útil cuando las conversaciones son largas.

3.2 El estado y sus alcances

El estado se organiza por el **prefijo de la clave**, que determina a quién pertenece cada valor y cuánto dura.

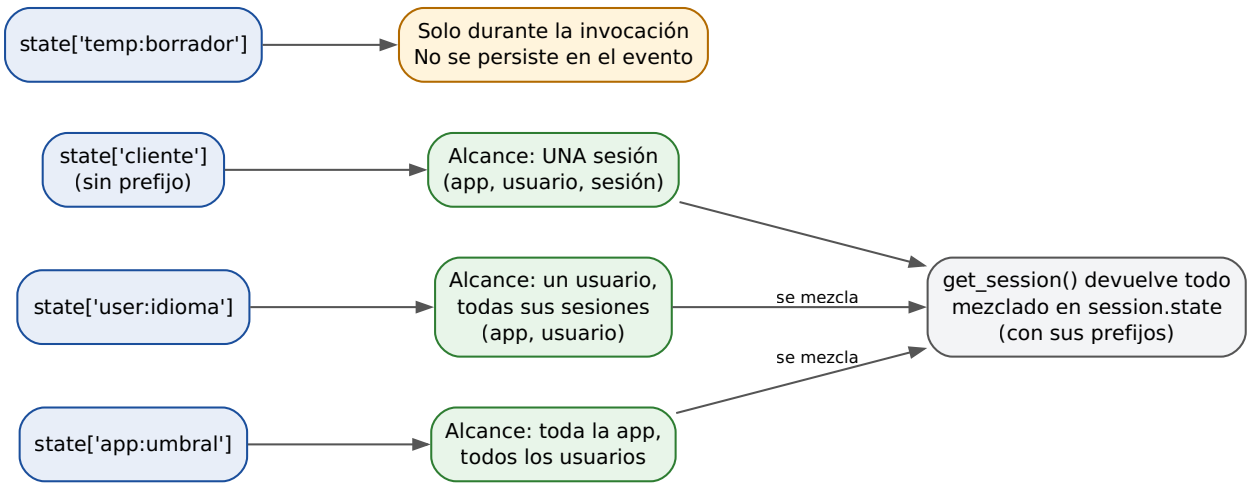


Figura 2 · Alcance del state: el prefijo de la clave decide a quién pertenece.

Una clave sin prefijo pertenece a esa sesión. El prefijo `user:` la comparte entre todas las sesiones del mismo usuario dentro de la aplicación; `app:` la comparte entre todos los usuarios de la aplicación; `temp:` la limita a la invocación en curso y no se persiste, porque se elimina del evento antes de guardarlo. Al leer una sesión, el servicio mezcla en `session.state` los valores de los alcances de aplicación y usuario, conservando sus prefijos. La documentación oficial precisa que las claves sin prefijo solo persisten con los servicios de base de datos o de Vertex AI.

Esto tiene una consecuencia arquitectónica valiosa: las claves `user:` constituyen una **forma de memoria entre sesiones sin embeddings ni búsqueda**. Para datos que se conocen de antemano —preferencias, idioma, datos de cliente, parámetros— es más barata, determinista y auditable que cualquier servicio de memoria.

Los cambios de estado viajan como `actions.state_delta` dentro de un evento, y el servicio de sesión los aplica en `append_event`. Hay tres maneras de producirlos. La habitual es asignar desde una herramienta o callback (`tool_context.state["clave"] = valor`), y el framework convierte la asignación en un delta. La segunda es `output_key` en la definición del agente, que guarda la respuesta final del

agente bajo esa clave. La tercera es proporcionar un estado inicial al crear la sesión. La documentación advierte contra modificar `session.state` directamente fuera de estos caminos, porque se omite el historial de eventos, se rompe la persistencia y se introducen problemas de concurrencia.

```python
def registrar_dato(campo: str, valor: str, tool_context: ToolContext) -> dict:
    tool_context.state[campo] = valor              # se convierte en state_delta
    tool_context.state["user:ultimo_campo"] = campo
    return {"ok": True}

agente = Agent(name="resumidor", model="...", instruction="...", output_key="resumen")
```

Para leer el estado desde el modelo hay dos vías: una plantilla `{variable}` en la instrucción, que ADK rellena desde el estado en el momento de armar la petición, o una herramienta que devuelva el dato. Que el modelo «sepa» algo porque aparece en el historial no implica que esté en el estado: son dos almacenes con dos lectores distintos, y confundirlos es una fuente habitual de errores.

3.3 Control del historial y compactación

El parámetro `include_contents` del agente admite `'default'`, en cuyo caso el modelo recibe el historial relevante, y `'none'`, en cuyo caso recibe solo la instrucción y la entrada actual. Para conversaciones que crecen sin límite, `App.events_compaction_config` acepta una `EventsCompactionConfig` con los campos `summarizer`, `compaction_interval`, `overlap_size`, `token_threshold` y `event_retention_size`, que permite resumir eventos antiguos. El procesador de compactación se ejecuta antes de armar los `contents`, de modo que el resumen ya forma parte de la petición. La compactación gestiona el *tamaño del historial dentro de una sesión*; no sustituye a la memoria de largo plazo, que opera entre sesiones.

3.4 El ciclo de una llamada al modelo

Entender el orden de operaciones es indispensable para razonar sobre cuándo se consulta la memoria y cuánto cuesta.

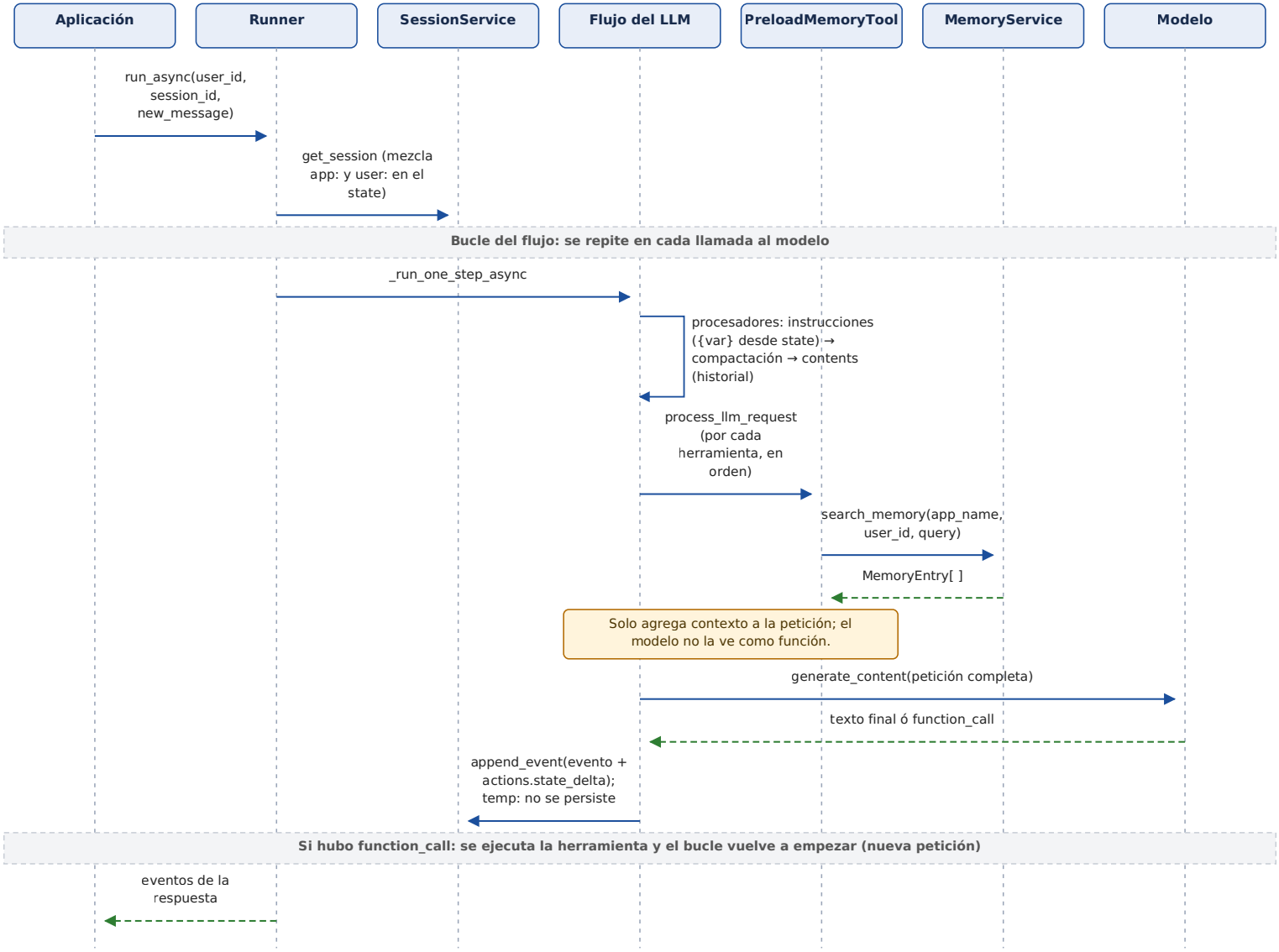


Figura 3 · Ciclo de una llamada al modelo: el flujo prepara la petición, ejecuta el preprocesamiento de las herramientas y recién entonces llama al modelo.

El `Runner` obtiene la sesión, mezclando el estado de aplicación y de usuario. El flujo del agente construye entonces la petición mediante una cadena de procesadores: básico, autenticación, confirmación, **instrucciones** (aplica las plantillas `{variable}` con el estado), identidad, **compactación**, **contents** (arma el historial) y otros. A continuación invoca `process_llm_request` en cada herramienta del agente, en el orden de `agent.tools`; es aquí donde `PreloadMemoryTool` consulta la memoria. Recién entonces se llama al modelo y se persiste el evento resultante, con su `state_delta`. Si el modelo solicitó una herramienta, esta se ejecuta y **el bucle vuelve al comienzo**, de modo que todo el preprocesamiento se repite en cada llamada al modelo dentro de la misma invocación.

4. Largo plazo: el servicio de memoria

4.1 El contrato

`BaseMemoryService` exige dos métodos y admite otros dos opcionales. `add_session_to_memory(session)` ingiere una sesión completa y puede invocarse varias veces durante su vida. `search_memory(app_name, user_id, query)` devuelve un `SearchMemoryResponse` con una lista de `MemoryEntry`. Los métodos opcionales son `add_events_to_memory(app_name, user_id, events, session_id, custom_metadata)`, que ingiere un **delta** de eventos sin reprocesar la sesión entera, y `add_memory(app_name, user_id, memories, custom_metadata)`, que escribe entradas ya elaboradas. Los servicios que no los soportan lanzan `NotImplementedError`, de modo que el código que los use debe conocer la implementación subyacente.

Una `MemoryEntry` contiene `content` (un `Content` de GenAI), `author`, `timestamp`, `id` y `custom_metadata`. Hay una ausencia notable: **el contrato no incluye borrar ni actualizar entradas**. Quien necesite derecho al olvido o corrección de datos debe resolverlo en su implementación o mediante los mecanismos del servicio administrado.

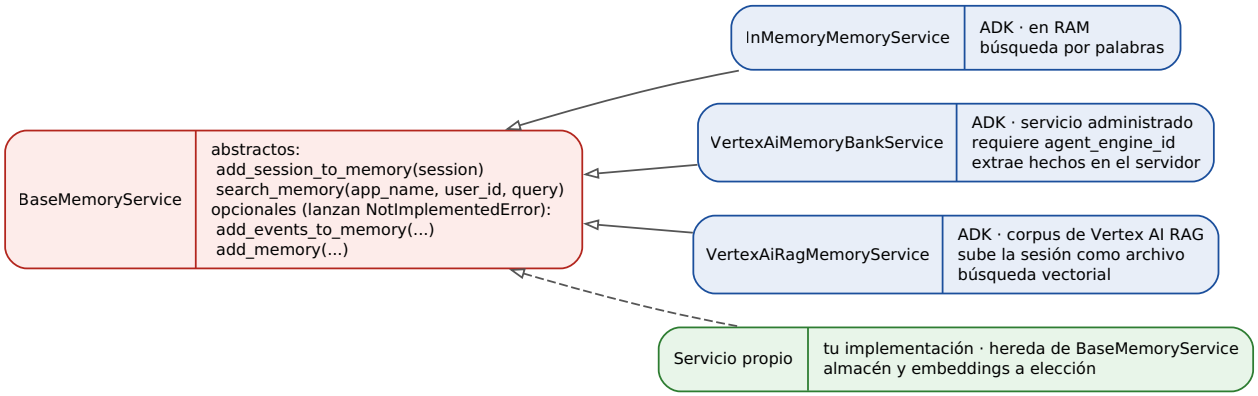


Figura 4 · Servicios de memoria: tres implementaciones incluidas en ADK y la posibilidad de una implementación propia sobre el mismo contrato.

4.2 La escritura: qué dispara el guardado

El flujo del `Runner` no contiene ninguna llamada automática a `add_session_to_memory`. La ingesta ocurre porque el código la pide, y hay tres lugares naturales para hacerlo: directamente sobre el servicio al cerrar una sesión; desde un callback, mediante el método homónimo del `Context`; o a través de los endpoints del servidor de ADK que lo invocan por cuenta del cliente. El patrón recomendado por la documentación es un `after_agent_callback`:

```python
async def guardar_sesion(callback_context):
    await callback_context.add_session_to_memory()

agente = Agent(name="a", model="...", after_agent_callback=guardar_sesion)
```

Elegir *cuándo* guardar es una decisión de diseño con consecuencias. Guardar la sesión completa al cerrarla es simple, pero deja la información fuera de alcance hasta que la sesión termina. Guardar un delta al final de cada turno (`add_events_to_memory`) la pone disponible antes y evita reprocesar lo ya ingerido, pero multiplica las operaciones de escritura. Escribir entradas elaboradas (`add_memory`) da control total sobre qué se recuerda, a costa de que el código decida qué es importante.

4.3 La lectura

`search_memory` devuelve entradas del usuario indicado. Qué significa «relevante» queda a cargo de la implementación: coincidencia de palabras, similitud vectorial o recuperación del lado del servidor. Quien llama —típicamente una herramienta— recibe la lista y decide qué hacer con ella.

5. Embeddings en la memoria de largo plazo

5.1 Por qué una memoria necesita embeddings

Una búsqueda por palabras clave falla de una manera previsible: encuentra lo que *dice lo mismo con las mismas palabras* y pierde lo que *significa lo mismo con otras*. Si un usuario mencionó en una conversación anterior que su empresa «evalúa financiar capital de trabajo», una consulta posterior sobre «líneas de crédito para operaciones corrientes» no comparte ninguna palabra relevante. Un **embedding** resuelve esto proyectando el texto a un vector de números reales en un espacio donde la cercanía geométrica aproxima la cercanía de significado. Textos con significado parecido quedan cerca; textos sobre temas distintos, lejos. Con ello la recuperación pasa de ser una coincidencia léxica a una búsqueda de vecinos más cercanos.

Para medir esa cercanía se usa habitualmente la **similitud coseno** o, cuando los vectores están normalizados a longitud uno, el **producto interno**, que es equivalente y más barato. Matemáticamente el coseno va de −1 a 1, aunque con textos del mismo idioma y dominio los valores útiles suelen concentrarse en la parte positiva. Algunos almacenes expresan la cercanía como **distancia** en lugar de similitud, y entonces el sentido se invierte: un valor menor es más cercano. Conviene verificar siempre en qué convención opera un servicio antes de fijar un umbral.

5.2 Dónde aparecen los embeddings en ADK

La respuesta depende de la implementación, y ese es uno de los criterios para elegir entre ellas.

`InMemoryMemoryService` **no usa embeddings**. Su búsqueda tokeniza la consulta y el texto de cada evento, cuenta las palabras en común y devuelve los eventos ordenados por esa cuenta, hasta un máximo de diez. Es una coincidencia léxica con puntuación simple, suficiente para probar un flujo, insuficiente para producción.

`VertexAiMemoryBankService` los usa **del lado del servidor, sin exponerlos**. Según la documentación de Memory Bank, las memorias se convierten en representaciones vectoriales para permitir búsqueda por similitud semántica, con un modelo de embeddings configurable y parámetros como `top_k` y filtros. El código de ADK no genera ni maneja vectores: en la lectura envía la consulta de texto como `similarity_search_params.search_query`, acotada al alcance `{app_name, user_id}`, y recibe memorias ya recuperadas. Para este servicio, el diseñador de la aplicación influye en los embeddings únicamente a través de la configuración de la instancia en Vertex AI.

`VertexAiRagMemoryService` los usa a través de **un corpus de Vertex AI RAG**. La sesión se sube como archivo al corpus, que la fragmenta, calcula embeddings y la indexa. La búsqueda se ejecuta con `retrieve_contexts` y acepta dos parámetros de control: `similarity_top_k`, la cantidad de contextos a recuperar, y `vector_distance_threshold`, que descarta los contextos cuya **distancia** vectorial supera el umbral. En ADK 2.10.0 el valor por defecto de ese umbral es 10, que en la práctica no filtra nada: quien necesite descartar resultados poco relevantes debe fijarlo explícitamente, calibrándolo contra el modelo de embeddings del corpus.

Una **implementación propia** puede usar cualquier modelo de embeddings y cualquier almacén vectorial. Es el único caso en el que el diseñador controla todos los parámetros —modelo, dimensionalidad, normalización, índice, umbrales—, y también el único en el que es responsable de todos ellos.

5.3 El ciclo de los embeddings: escribir y leer

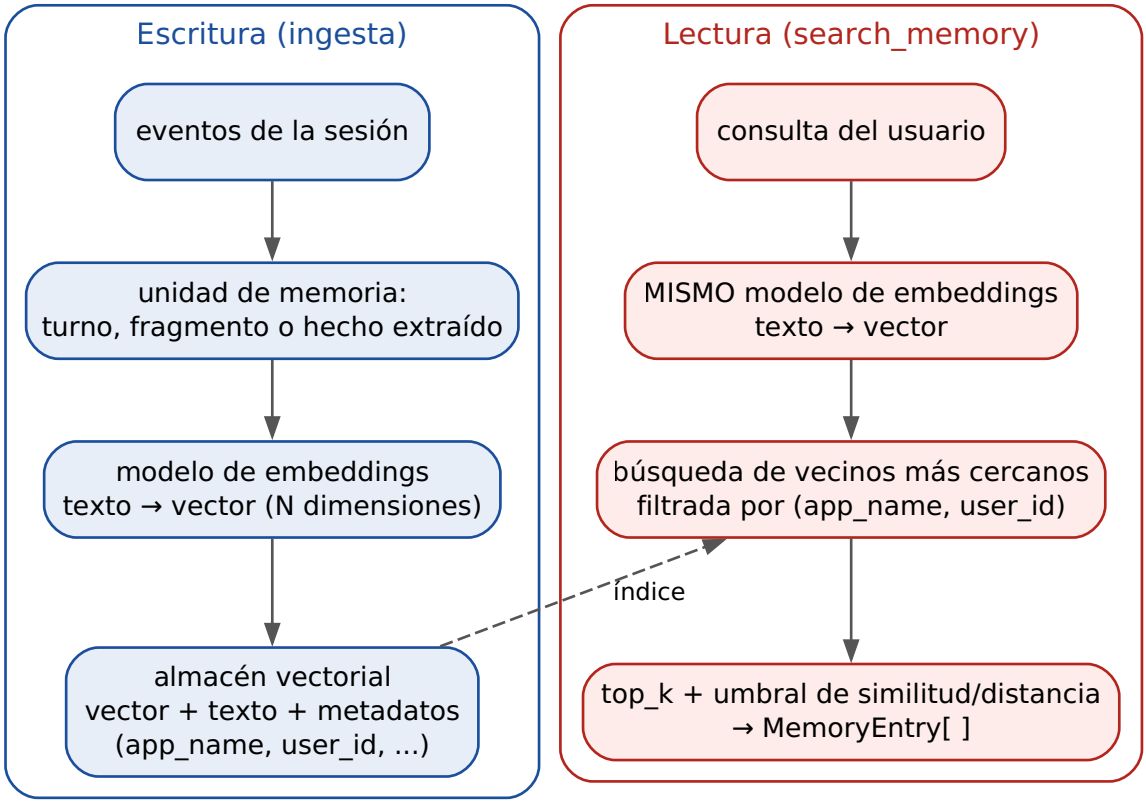


Figura 5 · Los embeddings en el ciclo de memoria: la misma transformación se aplica al guardar y al consultar.

Todo diseño de memoria vectorial se compone de dos recorridos que deben ser coherentes entre sí. En la **escritura**, los eventos de la sesión se transforman en una *unidad de memoria* —un turno, un fragmento de texto o un hecho extraído por un modelo—, se calcula su embedding y se almacena junto con el texto original y los metadatos de alcance. En la **lectura**, la consulta del usuario se transforma con el *mismo modelo*, se buscan los vecinos más cercanos restringidos al alcance `(app_name, user_id)` y se devuelven los mejores resultados, limitados por `top_k` y, si el almacén lo permite, por un umbral.

De esta simetría se derivan varias reglas operativas. El modelo de embeddings usado al consultar debe ser el mismo que se usó al indexar: mezclar modelos produce vectores incomparables y resultados sin sentido, y cambiar de modelo obliga a **reindexar todo el contenido**. La dimensionalidad y la normalización deben ser consistentes entre ambos recorridos. El filtro por usuario debe aplicarse en la búsqueda misma y no después, porque un filtrado posterior sobre un conjunto global de vecinos puede dejar a un usuario con menos resultados de los que le corresponden o, peor, depender de un control que alguien olvide. Y, por último, la calidad del resultado depende tanto de la unidad de memoria elegida como del modelo: un turno suelto pierde el contexto que lo rodea; un hecho consolidado es más preciso pero depende de la calidad de su extracción.

5.4 Consolidación: el triaje con embeddings

Los embeddings cumplen un segundo papel, menos evidente, en las memorias de **hechos consolidados** como Memory Bank. La consolidación resuelve un problema que los turnos crudos no resuelven: cuando un usuario corrige un dato («el plazo no es de 24 meses, es de 18»), una memoria

de turnos conserva ambos enunciados y deja al modelo la tarea de decidir cuál vale. Una memoria consolidada mantiene un único hecho vigente y conserva el anterior como historial.

Decidir si un hecho nuevo es un duplicado, una corrección o un dato independiente es una tarea que un modelo generativo resuelve bien, pero hacerlo para cada hecho encarecería y ralentizaría la escritura. El patrón habitual es un **triaje en dos etapas**: una comparación vectorial barata filtra los casos evidentes y reserva al modelo para los ambiguos.

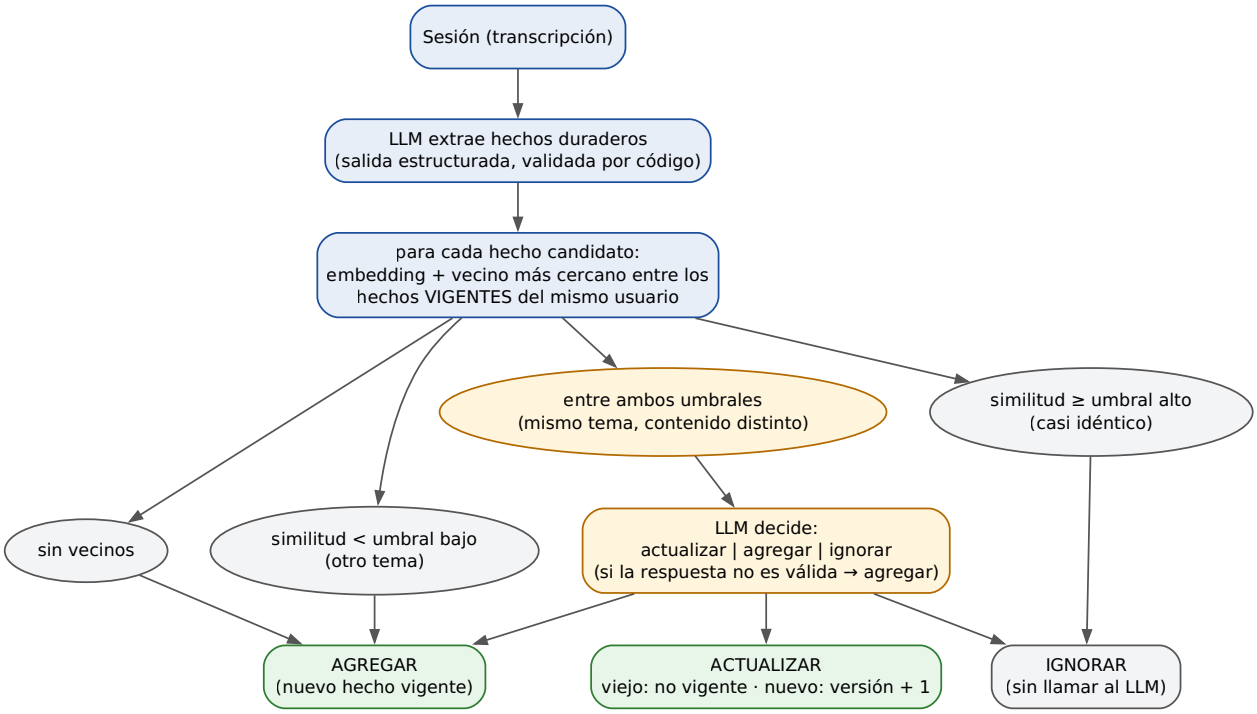


Figura 6 · Consolidación con triaje por embeddings: el LLM solo interviene en la zona intermedia.

El procedimiento es el siguiente. Un modelo extrae de la sesión los hechos duraderos, con una salida estructurada que el código valida. Para cada hecho candidato se calcula su embedding y se lo compara con el hecho **vigente** más cercano del mismo usuario. El resultado de esa comparación se interpreta con dos umbrales. Si la similitud supera el umbral alto, el hecho es prácticamente idéntico a uno existente y se descarta como duplicado, **sin invocar al modelo**. Si queda por debajo del umbral bajo, trata de un tema distinto y se agrega directamente, también sin invocar al modelo. Solo en la **zona intermedia** —mismo tema, contenido diferente, que es precisamente donde aparecen las correcciones— se le presenta al modelo el par de hechos para que decida entre actualizar, agregar o ignorar. El código valida esa respuesta y, si no es una de las tres acciones, aplica una por defecto (agregar). Una actualización marca el hecho anterior como no vigente y registra el nuevo con una versión incrementada y una referencia al que reemplaza; no se borra ninguna fila, de modo que el historial permanece consultable.

El resultado es un esquema **híbrido**: determinista en los extremos, probabilístico en el centro. Cada extremo ahorra una llamada al modelo, lo que reduce costo y latencia de escritura, y el código conserva la autoridad sobre las decisiones que no requieren juicio. Es, además, una buena ilustración del reparto de responsabilidades que conviene en sistemas con LLM: el modelo propone y desempata; el código valida, aplica reglas y controla el alcance.

Tres precisiones evitan malentendidos frecuentes sobre este patrón.

La primera es **cuándo se dispara**. La consolidación ocurre en la *escritura*, no en cada mensaje del usuario: se ejecuta cuando se ingiere una sesión o un delta, y se aplica una vez por cada hecho

extraído. Una sesión que no contiene hechos duraderos no genera ninguna comparación; una que contiene varios genera una por hecho. En Memory Bank esta lógica corre dentro del servicio administrado, que decide su propio disparo (por ejemplo, mediante la configuración de generación de memorias), y sus umbrales no son públicos; el esquema descrito es el patrón que se replica al construir un servicio propio.

La segunda es que **los umbrales dependen del modelo de embeddings y deben calibrarse**. La similitud entre dos formulaciones del mismo dato con un valor distinto («24 meses» frente a «18 meses») es alta, porque los embeddings capturan el tema y no la verdad ni el valor numérico; con modelos de embeddings habituales, pares de este tipo suelen caer en la zona intermedia y no en los extremos. Es lo deseable —una corrección no debe descartarse como duplicado—, pero significa que un umbral alto mal calibrado convierte correcciones en duplicados y las pierde en silencio. Los valores deben medirse con datos representativos del dominio y del idioma.

La tercera es que **un embedding no entiende negaciones ni cantidades**. «El cliente aprobó el crédito» y «el cliente no aprobó el crédito» producen vectores muy cercanos. Por eso la zona intermedia se confía a un modelo generativo y no a un tercer umbral: la comparación vectorial sirve para saber *de qué se habla*, no *si dos enunciados se contradicen*.

5.5 Consideraciones al incorporar embeddings propios

Quien construya una memoria vectorial propia debe decidir un conjunto de parámetros que los servicios administrados resuelven por su cuenta. El **modelo** determina el idioma, la calidad y el costo: un modelo compacto y local evita enviar texto a un tercero y reduce la latencia, mientras que un modelo hospedado ofrece mayor calidad a cambio de red y costo por llamada. La **dimensionalidad** afecta el almacenamiento y la velocidad de búsqueda. La **normalización** de los vectores permite usar el producto interno como coseno. El **índice** —exacto, para volúmenes pequeños, o aproximado, para volúmenes grandes— fija el compromiso entre exactitud y velocidad. Y el **almacén** puede ir desde un índice en proceso reconstruido al arrancar hasta una base de datos con extensión vectorial o un servicio de búsqueda vectorial administrado. La persistencia debe resolverse fuera de la RAM, y el índice debe poder reconstruirse a partir del almacén primario.

6. Los servicios de memoria incluidos en ADK

6.1 InMemoryMemoryService

Es la implementación de referencia para prototipos. Mantiene los eventos en un diccionario por `(app_name, user_id)` y por sesión, protegido por un candado, de modo que es seguro entre hilos. `add_session_to_memory` **reemplaza** lo guardado bajo ese identificador de sesión, por lo que ingerir la misma sesión dos veces no duplica. `add_events_to_memory` **mezcla** por `event.id`. En la búsqueda se tokeniza con soporte Unicode, se cuenta cuántas palabras de la consulta coinciden con cada evento, se ordena de mayor a menor coincidencia con orden estable y se devuelve como máximo diez resultados. No hay semántica ni persistencia; su valor está en que permite ejercitar el flujo completo sin infraestructura.

6.2 VertexAiMemoryBankService

Memory Bank es un servicio administrado de Vertex AI que **extrae memorias** a partir de los eventos, las consolida con las existentes y las recupera por similitud semántica. Requiere un Agent Engine, cuyo identificador se pasa como `agent_engine_id` (obligatorio, y debe ser solo el identificador, no la ruta completa del recurso); el servicio de memoria no exige desplegar el agente allí. La autenticación usa credenciales de aplicación por defecto, aunque admite credenciales explícitas o una clave de modo Express.

La **escritura** tiene tres caminos con semánticas distintas. Por defecto, la ingesta de eventos usa la API `memories.ingest_events`, y lo hace como **tarea en segundo plano**: la llamada a `add_session_to_memory` retorna sin esperar a que la memoria esté generada, y los errores quedan registrados en el log y no se propagan. Esto implica consistencia eventual: una consulta inmediatamente posterior a la ingesta puede no ver aún las memorias nuevas. Si se pasan en `custom_metadata` claves que solo admite la API de generación (`ttl`, `revision_ttl`, `metadata`, `wait_for_completion`, `allowed_topics`, `disable_consolidation`, entre otras), se usa `memories.generate`, cuya llamada se aguarda en el código (y, con `wait_for_completion`, hasta que la generación termina) y que permite, por ejemplo, restringir la generación a determinados temas (`allowed_topics`) o desactivar la consolidación. Para la ingesta incremental, el modo por defecto admite además `stream_id`, `force_flush` y una `generation_trigger_config` que controla cuándo se genera (por ejemplo, tras un período de inactividad). Los eventos sin contenido útil se filtran antes de enviarse.

Para entradas ya elaboradas, `add_memory` escribe directamente con `memories.create`, una llamada por memoria y sin intervención del modelo, o bien, si `custom_metadata["enable_consolidation"]` es verdadero, usa `memories.generate` con una fuente de memorias directas, en lotes de cinco, de modo que el servicio las consolide con las existentes. Si la entrada trae `id`, este se reenvía como identificador de la memoria, lo que permite al llamador controlar el nombre del recurso.

La **lectura** invoca `memories.retrieve` con alcance `{app_name, user_id}` y una búsqueda por similitud sobre la consulta. Cada memoria recuperada se convierte en una `MemoryEntry` de autor `user` cuyo texto es el hecho, con la hora de actualización y los metadatos. Debe notarse que `search_memory` solo envía el texto de la consulta: parámetros de la API como `top_k` o filtros por metadatos no se exponen a través de esta interfaz, y quien los necesite debe invocar la API de Vertex AI directamente o envolver el servicio en uno propio. ADK ofrece además `retrieve_profiles`, que devuelve perfiles estructurados del usuario según los esquemas registrados en la instancia, una forma de memoria más cercana a un registro que a una colección de hechos sueltos.

Entre sus capacidades de gobierno, la documentación de Memory Bank menciona alcances y temas, TTL para expiración automática, revisiones que registran el cambio de las memorias a lo largo del tiempo y perfiles.

6.3 VertexAiRagMemoryService

Esta implementación trata la memoria como un problema de recuperación sobre transcripciones. Cada sesión se serializa como un archivo de texto con una línea JSON por evento (autor, marca temporal y texto) y se sube a un corpus de Vertex AI RAG, con un nombre de archivo que codifica `app_name`, `user_id` y `session_id`. El corpus fragmenta, calcula embeddings e indexa.

Al buscar, el servicio primero intenta **acotar la búsqueda a los archivos del usuario**: lista los archivos del corpus, selecciona los que pertenecen al `app_name` y `user_id` solicitados y consulta solo esos. Ese listado está limitado a diez páginas de cien archivos; si el corpus es mayor y no se completa, el servicio registra una advertencia y consulta sin acotar. Por eso, después de recuperar, **siempre vuelve a filtrar del lado del cliente** por alcance, de modo que el aislamiento entre usuarios se conserva aun cuando la búsqueda no se haya podido acotar, con el costo de transferir y descartar resultados ajenos. En la salida, los fragmentos recuperados se reconstruyen como eventos, se fusionan los solapados dentro de una misma sesión y se ordenan por tiempo.

Los parámetros del constructor son `rag_corpus`, `similarity_top_k` y `vector_distance_threshold`, además de `project` y `location`. Requiere el paquete de Vertex AI instalado (`google-adk[gcp]`). Su caso de uso natural es una infraestructura RAG preexistente o la necesidad de recuperar transcripciones completas con su contexto, no hechos destilados.

6.4 Comparación

	`InMemoryMemoryService`	`VertexAiMemoryBankService`	`VertexAiRagMemoryService`
Qué guarda	Eventos originales	Hechos extraídos y consolidados	Transcripciones en un corpus
Embeddings	No	Sí, internos al servicio	Sí, internos al corpus
Búsqueda	Palabras en común	Similitud semántica	Similitud vectorial
Escritura	Síncrona, en RAM	Asíncrona por defecto	Subida de archivo
Persistencia	No	Sí	Sí
Mejor uso	Prototipos y pruebas	Memoria de usuario con hechos que evolucionan	Transcripciones sobre infraestructura RAG existente

7. Cómo llega la memoria al modelo

Recuperar y mostrar lo recuperado son dos pasos distintos. `search_memory` devuelve datos *al código*; hace falta una herramienta o un callback que los incorpore a la petición. ADK ofrece dos herramientas integradas y deja abierta la posibilidad de escribir la propia.

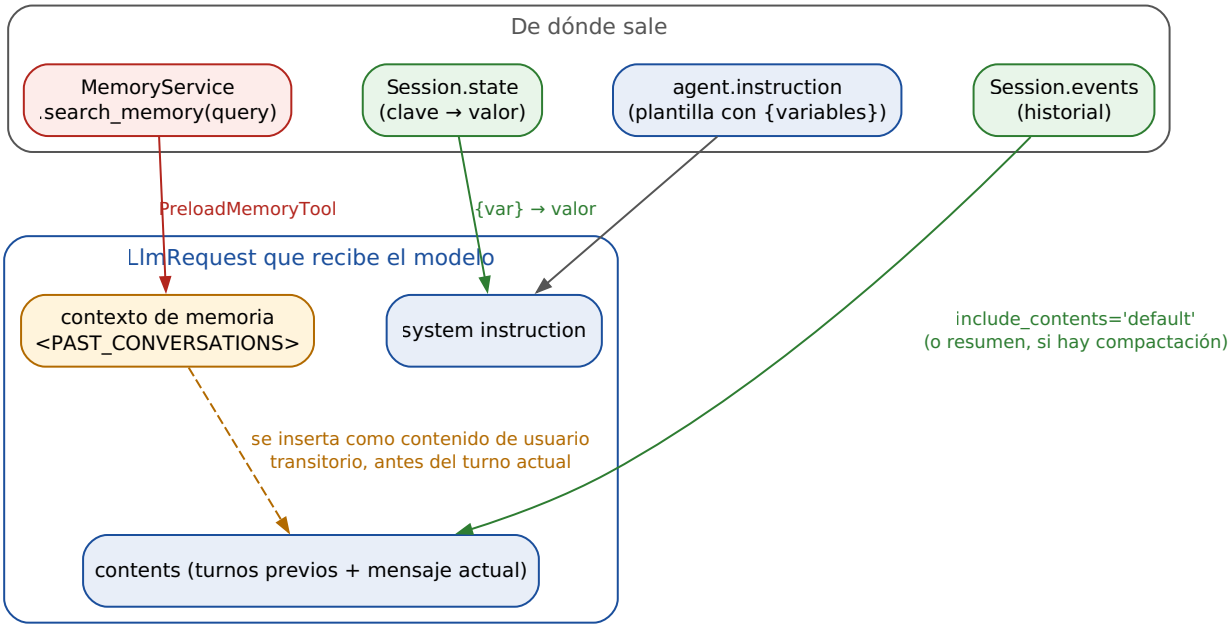


Figura 7 · Qué entra en la petición al modelo y de dónde sale.

7.1 PreloadMemoryTool: recuperación automática

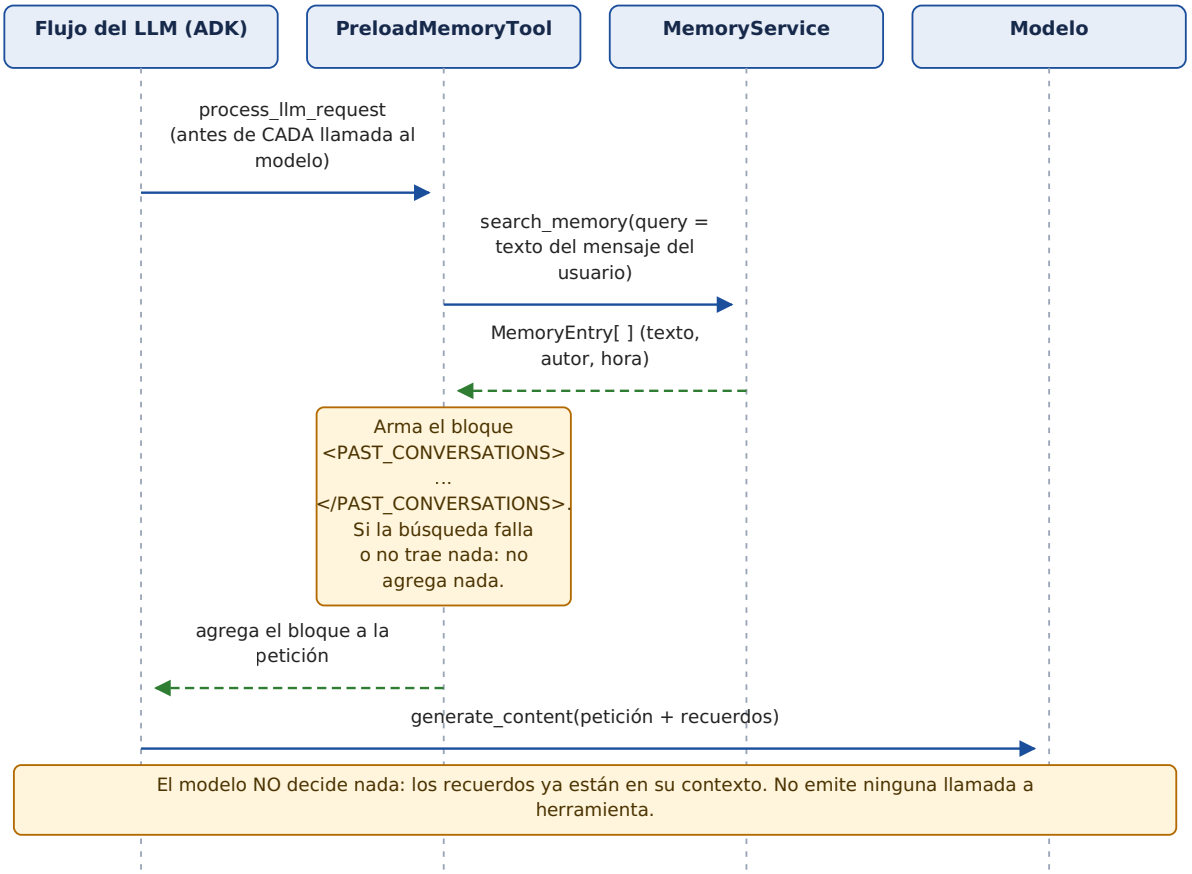


Figura 8 · PreloadMemoryTool: se ejecuta antes de cada llamada al modelo y agrega los recuerdos a la petición. El modelo nunca la ve como función.

`PreloadMemoryTool` solo implementa `process_llm_request`: modifica la petición y **no declara ninguna función** que el modelo pueda invocar. Se ejecuta automáticamente antes de cada llamada. La consulta que envía a la memoria es el texto del **mensaje del usuario de la invocación** —todas las partes de texto, concatenadas—, no la conversación completa. Con las memorias recuperadas arma un bloque con el siguiente formato, usando únicamente las partes de texto de cada entrada:

```
The following content is from your previous conversations with the user.
They may be useful for answering the user's current query.
<PAST_CONVERSATIONS>
Time: <marca de tiempo>
<autor>: <texto>
...
</PAST_CONVERSATIONS>
```

En ADK 2.10.0 el bloque se inserta como un **contenido de usuario transitorio**, en el límite del turno actual (antes del último mensaje del usuario, o después de la respuesta de una herramienta si el modelo continúa un turno de herramientas). El contenido es transitorio porque se agrega solo a esa petición y no se guarda en el historial. La razón de diseño es mantener el contenido propio de cada petición fuera de un prefijo reutilizable —la instrucción del sistema y el historial—, lo que preserva la eficacia del caché de contexto del modelo. Una consecuencia práctica: cualquier prueba o instrumentación que busque el bloque `<PAST_CONVERSATIONS>` en la instrucción del sistema no lo encontrará; hay que inspeccionar la petición completa.

Su comportamiento ante fallos es deliberadamente tolerante. Si `search_memory` lanza una excepción, la herramienta no la propaga: registra una advertencia con la longitud de la consulta y la traza, y la petición continúa sin memoria. Si no hay resultados, no agrega nada. Esta tolerancia es útil para la

disponibilidad, pero oculta el problema a quien no monitoree los logs, por lo que conviene instrumentarla (sección 10.5). Como el preprocesamiento corre en cada paso del bucle, la consulta a la memoria se repite en cada llamada al modelo dentro de la invocación *(inferido)*.

7.2 LoadMemoryTool: el modelo decide

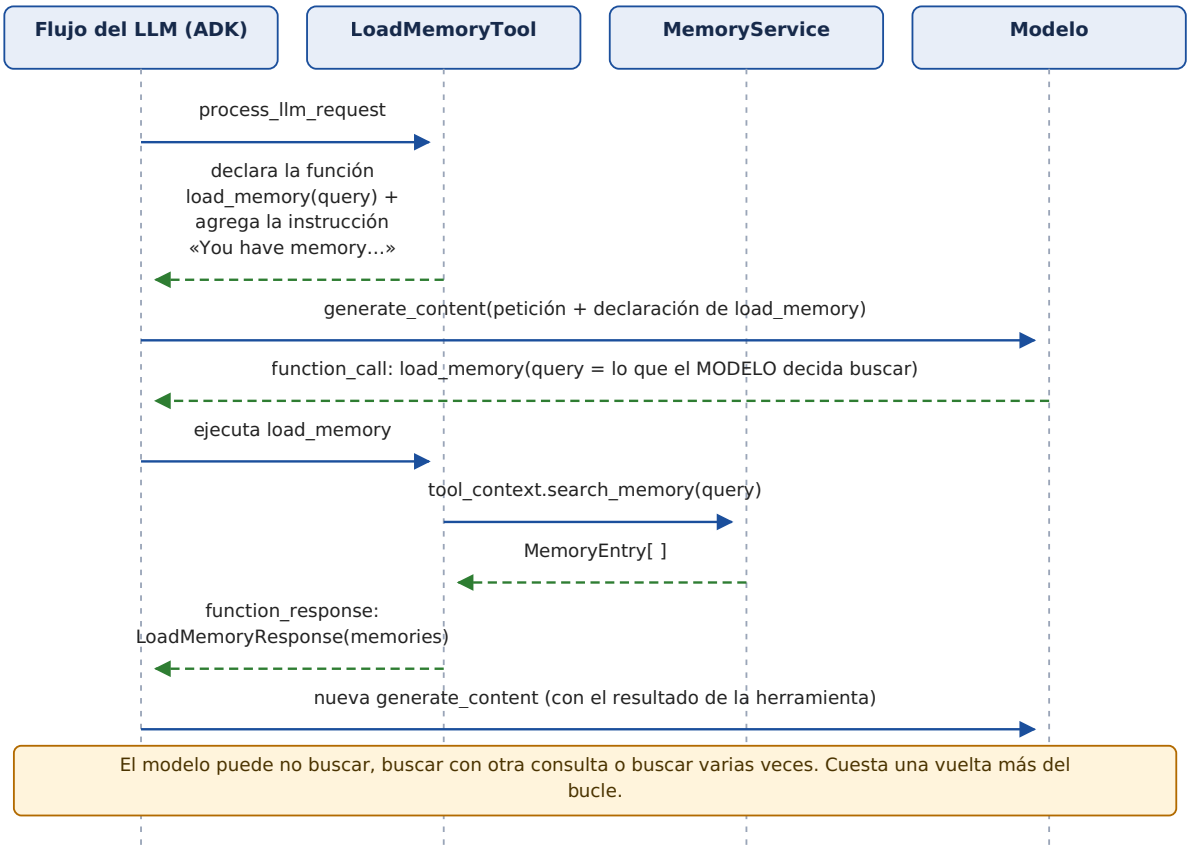


Figura 9 · LoadMemoryTool: el modelo decide si buscar y con qué consulta; cada búsqueda cuesta una vuelta más del bucle.

`LoadMemoryTool` es una herramienta de función que expone `load_memory(query)` al modelo. Al preparar la petición agrega una instrucción que le informa de que dispone de memoria y de que debe invocar esa función con una consulta cuando una pregunta lo requiera. El modelo elige la consulta, puede no buscar, o buscar varias veces. A cambio, cada búsqueda cuesta una llamada adicional al modelo, porque la solicitud y su respuesta forman parte del bucle. La respuesta es un `LoadMemoryResponse` con la lista de memorias.

7.3 Lectura propia

Cualquier herramienta o callback puede invocar `await ctx.search_memory(query)` y decidir qué hacer con el resultado: filtrarlo, reordenarlo, resumirlo, combinarlo con otras fuentes o devolverlo como respuesta de herramienta. Un `before_model_callback` recibe la petición ya armada, lo que permite inspeccionarla o modificarla. Es la opción adecuada cuando la consulta literal del usuario es un mal criterio de búsqueda —por ejemplo, un mensaje de seguimiento como «¿y el otro?», que por sí solo no recupera nada útil—, y se quiere reformularla con el contexto de la conversación antes de consultar.

7.4 Cómo elegir

`PreloadMemoryTool` conviene cuando la memoria casi siempre aporta y no se quiere depender de que el modelo decida consultarla. Su riesgo principal es la calidad de la consulta: la recuperación depende

de las palabras del mensaje actual. `LoadMemoryTool` conviene cuando la memoria es grande y solo a veces hace falta, y se acepta el costo de una vuelta adicional; su riesgo principal es que el modelo **no busque** o busque mal. Una herramienta propia es la salida cuando se necesita filtrar, reordenar o combinar resultados, al costo de mantener más código. Los tres enfoques pueden combinarse: es habitual precargar el contexto básico del usuario y dejar `load_memory` para consultas puntuales.

8. Servicios de memoria propios

Heredar de `BaseMemoryService` e implementar los dos métodos abstractos alcanza para integrar cualquier almacén con las herramientas de ADK:

```python
from google.adk.memory import BaseMemoryService
from google.adk.memory.base_memory_service import SearchMemoryResponse
from google.adk.memory.memory_entry import MemoryEntry

class MiMemoria(BaseMemoryService):
    async def add_session_to_memory(self, session) -> None:
        ...   # recorrer session.events y almacenar lo que corresponda

    async def search_memory(self, *, app_name: str, user_id: str, query: str) -> SearchMemoryResponse:
        ...   # devolver SearchMemoryResponse(memories=[MemoryEntry(...), ...])
```

Se justifica un servicio propio cuando hay requisitos que los servicios incluidos no cubren: residencia de datos, un almacén vectorial ya adoptado por la organización, políticas de borrado específicas, control fino de umbrales y `top_k`, o una lógica de consolidación propia. Dos arquitecturas de referencia cubren la mayoría de los casos.

8.1 Almacén de turnos con índice vectorial

Es el diseño más simple. Cada turno de la conversación se guarda tal cual, con su embedding, en un almacén persistente; un índice vectorial acelera la búsqueda y se reconstruye desde el almacén al arrancar. La escritura es determinista y no involucra a ningún modelo generativo: solo cuesta el cálculo de embeddings. La lectura calcula el embedding de la consulta, restringe la búsqueda al alcance del usuario y devuelve los `top_k` turnos más parecidos, idealmente con la puntuación en `custom_metadata`. La idempotencia se logra con una clave de unicidad sobre el turno (alcance, sesión, autor y texto), de modo que reingerir la misma sesión no duplica.

Su fortaleza es la fidelidad —el modelo recibe lo que realmente se dijo— y su debilidad es doble. Las correcciones se acumulan: la memoria contiene el dato viejo y el nuevo, y ambos pueden recuperarse. Y todo depende de la lectura: si el dato relevante no entra entre los `top_k` más parecidos a la consulta, el modelo no lo ve, aunque esté guardado.

8.2 Almacén de hechos consolidados

Es el diseño de Memory Bank, replicado localmente. En lugar de guardar la conversación, un modelo extrae hechos duraderos y el servicio los consolida con el procedimiento de la sección 5.4. El esquema de almacenamiento típico incluye el hecho, su embedding, la sesión de origen, una versión, un indicador de vigencia y una referencia al hecho que reemplaza. La lectura considera solo hechos vigentes, calcula la similitud, descarta lo que queda por debajo de un piso mínimo y devuelve hasta `top_k`. Aquí el borrado por usuario es sencillo de ofrecer, y el historial de versiones da trazabilidad.

La fortaleza es una lectura limpia y la gestión de correcciones; la debilidad es que el riesgo se traslada a la escritura. Una extracción incorrecta se almacena como si fuera un hecho, y una corrección redactada de manera muy distinta puede no detectarse como corrección. Cuesta, además, llamadas a un modelo al escribir.

8.3 Criterios de diseño que el contrato no impone

El contrato deja sin resolver varios aspectos que una implementación propia debe decidir de forma consciente. El **alcance** debe filtrarse siempre por `(app_name, user_id)`; ADK no lo hace por la implementación. La **idempotencia** exige que guardar la misma sesión dos veces no duplique. La **persistencia** requiere almacenar fuera de la RAM y poder reconstruir los índices. Los **metadatos**

(`custom_metadata`) son el lugar para puntuaciones, versiones u origen de la memoria. Los **fallos** de `search_memory` son capturados por `PreloadMemoryTool` sin propagarse, por lo que la implementación debe registrar sus propios errores. El **borrado y la corrección** no existen en el contrato y hay que agregarlos si el caso de uso los necesita. Y la **utilidad de la lectura** depende de devolver pocos resultados y relevantes, porque todo lo que se devuelve termina en el contexto del modelo.

La división de responsabilidades que mejor funciona cuando hay un modelo generativo en la escritura es la ya descrita: el modelo propone —extrae, desempata—, y el código decide —valida las salidas estructuradas, aplica los umbrales, restringe el alcance y borra—. Cuando el modelo debe producir JSON, conviene pasarlo como salida estructurada con esquema, volver a validarlo en el código, reintentar un número acotado de veces ante una salida inválida y, si persiste, **no guardar nada** y dejar constancia, en lugar de almacenar un dato dudoso. Con modelos que usan razonamiento interno, es importante verificar que esos tokens no consuman el límite de salida y dejen el JSON cortado.

9. Patrones de uso y selección

La regla general es empezar por el mecanismo más simple que resuelva la necesidad. El estado con prefijo `user:` suele bastar cuando los datos se conocen de antemano; un servicio de memoria con búsqueda recién se justifica cuando hay que recuperar conversaciones pasadas en lenguaje natural.

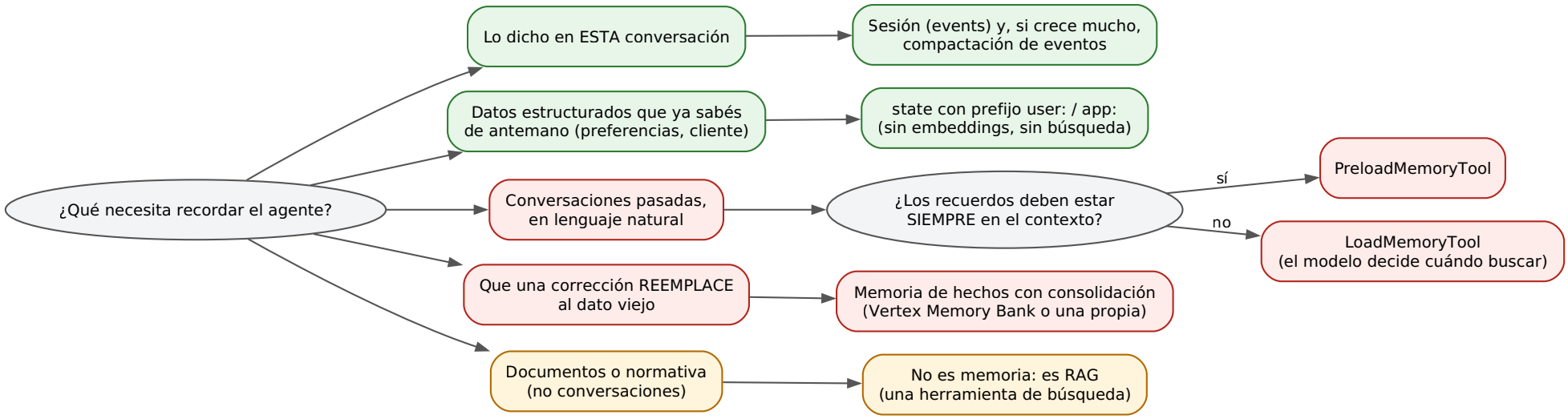


Figura 10 · Cómo elegir: según lo que el agente necesita recordar.

Los patrones habituales, de menos a más complejos, son cinco. El primero es **solo sesión**: una conversación, sin memoria entre sesiones, que es el valor por defecto. El segundo es **estado entre sesiones**, con claves `user:` y `app:`: barato, determinista y suficiente cuando los datos son conocidos. El tercero son **turnos crudos con búsqueda vectorial**, donde la simplicidad se paga con riesgo en la lectura y acumulación de correcciones. El cuarto son **hechos consolidados**, con lectura limpia y manejo de correcciones, al precio de llamadas a un modelo en la escritura y riesgo de extracción. El quinto es **RAG sobre documentos**, que no es memoria del usuario pero suele combinarse con ella: la memoria dice quién es el usuario y qué se habló, y el RAG dice qué establece la normativa o la documentación.

La necesidad de que una corrección reemplace al dato viejo es, en la práctica, el criterio que más a menudo decide entre turnos crudos y hechos consolidados. Si los datos del usuario evolucionan —plazos, montos, preferencias— y el agente debe actuar sobre el valor vigente, los turnos crudos exigen que el modelo reconcilie versiones contradictorias en cada respuesta, algo que no debería delegarse en algo tan variable como una generación.

Los modos de falla de cada diseño son distintos y conviene tenerlos presentes al diseñar pruebas. Con turnos crudos el riesgo es la **lectura**: un dato guardado que no llega al modelo, mitigable con un `top_k` mayor, consultas más ricas, filtros o `LoadMemoryTool`. Con hechos consolidados el riesgo es la **escritura**: una extracción incorrecta o una corrección no detectada, mitigable con validación de la salida, revisión de umbrales y conservación del historial. Con `PreloadMemoryTool` el riesgo es la **consulta**, que depende de las palabras del mensaje actual; se mitiga combinándolo con una herramienta de lectura propia. Con `LoadMemoryTool` el riesgo es que el **modelo no busque**, y se mitiga con instrucciones claras o precargando. Esta descripción corresponde al diseño; no pretende medir con qué frecuencia ocurre cada falla en un caso concreto, algo que solo se establece evaluando con datos propios.

10. Consideraciones de producción

10.1 Aislamiento y alcance

El contrato indexa por `(app_name, user_id)`, pero el cumplimiento es responsabilidad de la implementación. Memory Bank lo resuelve en el servicio mediante el alcance. El servicio basado en RAG lo hace a través del nombre del archivo, con acotación previa y un filtro final del lado del cliente que lo mantiene incluso cuando la acotación falla. Las implementaciones propias deben aplicarlo a mano y, preferentemente, dentro de la consulta vectorial. Probar siempre que un `user_id` distinto no recupere memoria ajena, y que un usuario sin memoria reciba una lista vacía y no un error ni resultados de otros.

10.2 Privacidad, retención y borrado

`BaseMemoryService` no tiene método de borrado, por lo que el derecho al olvido, las correcciones y las políticas de retención deben resolverse en el servicio. Memory Bank ofrece TTL mediante `custom_metadata` (`ttl`, `revision_ttl`) y revisiones para trazabilidad. Una implementación propia debe ofrecer, como mínimo, el borrado completo por usuario, incluyendo versiones históricas y vectores. Además, las consultas y los pasajes que procesan los servicios de Vertex AI salen del proceso hacia Google Cloud: cuando los datos son sensibles, hay que verificar antes la región, el acuerdo de procesamiento de datos y el control de acceso del destino. Los embeddings no son datos anónimos: pueden revelar información del texto original y deben tratarse con la misma política que este.

10.3 Consistencia y latencia de escritura

La ingesta en Memory Bank es asíncrona por defecto, de modo que la memoria nueva puede no estar disponible de inmediato. Si un flujo exige leer lo que se acaba de escribir —por ejemplo, un usuario que corrige un dato y pregunta enseguida—, hay que usar la ruta de generación con `wait_for_completion` o diseñar el flujo para tolerar la demora. Las consolidaciones con modelo agregan latencia de escritura que conviene sacar del camino crítico de la conversación, por ejemplo ejecutándolas al cerrar la sesión.

10.4 Costo y tamaño del contexto

Con `PreloadMemoryTool` la consulta a la memoria se repite en cada llamada al modelo, de modo que un servicio costoso (embeddings, red) multiplica su costo por las vueltas del bucle. En la escritura, los diseños con modelo generativo pagan al guardar, mientras que los de turnos crudos solo pagan embeddings. Y todo lo que devuelve `search_memory` entra en el contexto: devolver muchas entradas encarece, aumenta la latencia y puede diluir la respuesta. Por eso conviene tratar `top_k` y los umbrales como parámetros de diseño y no como valores por defecto.

10.5 Observabilidad y pruebas

Es necesario distinguir lo que recuperó el servicio de lo que recibió el modelo. Un `before_model_callback` que registre la petición permite comprobar qué recuerdos llegaron efectivamente, y es la verificación que realmente importa. Para probar sin invocar a un modelo, `PreloadMemoryTool.process_llm_request` puede llamarse con un contexto mínimo —algo que exponga `user_content` y `search_memory`— y una `LlmRequest` vacía, lo que permite pruebas deterministas de la inyección. Además de la ruta feliz, conviene probar el aislamiento entre usuarios, la lista vacía y el comportamiento ante un `search_memory` que falla, y monitorear las advertencias que `PreloadMemoryTool` registra al absorber excepciones.

10.6 Evaluación de la calidad de recuperación

Una memoria que «parece funcionar» en una demostración puede fallar en producción por razones que no se ven a simple vista. Conviene evaluar la recuperación como un componente propio: armar un conjunto de pares consulta–memoria esperada, medir qué proporción de las veces aparece la memoria correcta entre los `k` resultados, y repetir la medición al cambiar el modelo de embeddings, el `top_k`, los umbrales o la estrategia de consulta. En diseños con consolidación, medir además la tasa de duplicados no detectados, de correcciones perdidas y de hechos mal extraídos, porque son los errores que se acumulan silenciosamente.

11. Glosario

Sesión. Una conversación: su historial (`events`) y su estado (`state`).

Evento. Unidad del historial: un mensaje, una llamada a herramienta o su respuesta, con un `actions.state_delta` opcional.

`state_delta`. Cambios de estado que viajan dentro de un evento y se aplican en `append_event`.

Alcance `(app_name, user_id)`. Clave con la que se separa la memoria de cada usuario y aplicación.

`MemoryEntry`. Entrada devuelta por la memoria: contenido, autor, hora, identificador y metadatos.

Ingesta. Pasar una sesión, un delta de eventos o entradas elaboradas a la memoria de largo plazo.

Embedding. Representación vectorial de un texto en la que la cercanía aproxima la similitud de significado.

Similitud coseno / producto interno. Medidas de cercanía entre vectores; equivalentes si los vectores están normalizados.

`top_k`. Cantidad máxima de resultados que devuelve una búsqueda.

Consolidación. Decidir si un hecho nuevo se agrega, reemplaza a otro o se ignora.

Hecho vigente. Hecho que sigue valiendo; los reemplazados se conservan como historial.

Triaje. Filtrado barato que resuelve los casos evidentes y reserva la decisión costosa para los ambiguos.

Contenido transitorio. Contenido de usuario que ADK agrega solo a una petición, sin guardarlo en el historial.

Memoria episódica y declarativa. Analogía de la ciencia cognitiva que distingue guardar episodios (turnos) de guardar aserciones (hechos). ADK no implementa estas categorías; son una forma de pensar lo que cada diseño almacena.

12. Referencias

- Documentación de ADK sobre sesiones y memoria: https://adk.dev/sessions/ y https://adk.dev/sessions/memory/.
- Documentación de ADK sobre estado de sesión: https://adk.dev/sessions/state/.
- Documentación de Memory Bank en Google Cloud: https://docs.cloud.google.com/agent-builder/agent-engine/memory-bank/overview.
- Código fuente de ADK 2.10.0: paquetes `google.adk.memory`, `google.adk.sessions`, `google.adk.tools` (`preload_memory_tool`, `load_memory_tool`) y `google.adk.agents.context`.

Apéndice. Método de verificación y límites

Las clases y funciones citadas se revisaron en el código fuente de ADK 2.10.0: contratos y firmas de `BaseMemoryService` y `Session`; el código de `InMemoryMemoryService`, `VertexAiMemoryBankService`, `VertexAiRagMemoryService`, `PreloadMemoryTool` y `LoadMemoryTool`; los campos de `EventsCompactionConfig` y `MemoryEntry`; los métodos del `Context`; y los esquemas de URI registrados para los servicios de memoria. Las afirmaciones sobre Memory Bank, su uso de embeddings y sus capacidades de gobierno provienen de la documentación oficial y no se contrastaron ejecutando el servicio contra la nube.

No se verificó el comportamiento fino de la compactación de eventos, el almacenamiento interno de `DatabaseSessionService` y `VertexAiSessionService`, ni el comportamiento de los servicios administrados de Vertex AI en operación real. Los umbrales de similitud dependen del modelo de embeddings y del idioma, y deben calibrarse con datos propios. Las versiones posteriores de ADK pueden cambiar los comportamientos descritos; ante una actualización, conviene reinspeccionar `PreloadMemoryTool`, `InMemoryMemoryService` y los servicios de Vertex AI.